

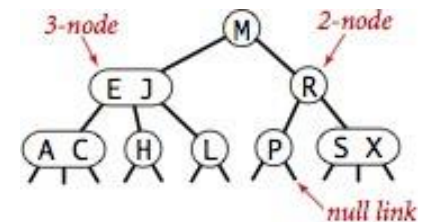


ALBERI DI RICERCA 2-3

Martedì 24 Marzo 2026

ALBERO DI RICERCA 2-3 (MAGGIORE FLESSIBILITÀ NEL GRADO DEI NODI)

- Sono stati introdotti da Hopcroft in un articolo mai pubblicato e poi descritti in Aho,Hopcroft,Ullman, Data Structures and Algorithms, 1983.
- Un Albero di Ricerca 2-3 (2-3ST) è un albero che, se non è vuoto, è
 - Un 2-nodo a una chiave, con link sinistro a un 2-3ST con chiavi più piccole e link destro a un 2-3ST con chiavi più grandi;
 - Un 3-nodo a due chiavi, con link sinistro a un 2-3ST con chiavi più piccole, link centrale a un 2-3ST con chiavi comprese tra le due, e link destro a un 2-3ST con chiavi più grandi.
- Ovvero, in un albero di ricerca 2-3 ogni nodo interno ha 2 o 3 figli.
- Un link ad un albero vuoto è detto link nullo.
- Un nodo esterno non serve a memorizzare alcun dato, ha solo un ruolo di «segnaposto». Posso essere rimpiazzati da link nulli.
- Usiamo 2-3ST che sono perfettamente bilanciati: tutti i link nulli hanno la stessa distanza dalla radice.
- Se ci sono n chiavi nel 2-3 ST, il numero dei link nulli è pari a $n+1$ (dimostrarlo per esercizio: suggerimento...usare l'induzione)
- Una visita in ordine simmetrico produce una lista delle chiavi in ordine non decrescente

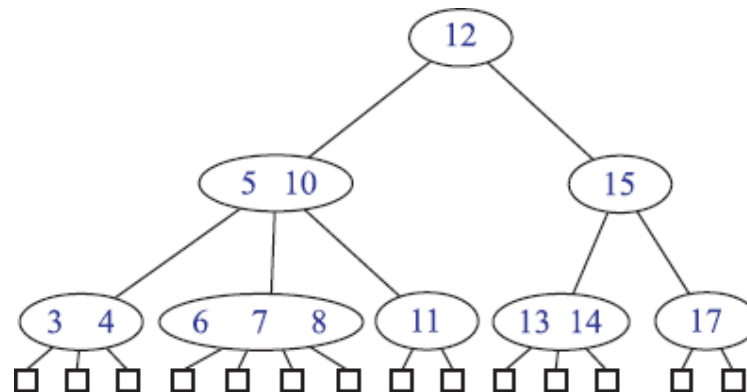


Anatomy of a 2-3 search tree



OSSERVAZIONI

- Esistono due possibili interpretazioni (e quindi utilizzi degli alberi 2-3):
 1. Le informazioni sono memorizzate nelle foglie. I nodi interni contengono copie di alcune informazioni contenute nelle foglie. In questo caso le chiavi contenute nelle foglie appaiono in ordine crescente da sinistra verso destra
 2. Un caso particolare degli alberi di ricerca a più vie: un altro caso studiato sono gli alberi (2,4) o (2,3,4). Ogni nodo contiene informazioni
- Qui considereremo la seconda interpretazione. Il caso 1 lo tratteremo studiando i B-trees



RELAZIONE TRA NODI E ALTEZZA NEGLI ALBERI 2-3

- Teorema: Sia T un albero con 2-3 nodi con n voci, f foglie e altezza h . Le seguenti diseguaglianze sono soddisfatte da n , f e h :

- $2^{h+1} - 1 \leq n \leq \frac{3^{h+1} - 1}{2}$

- $2^h \leq f \leq 3^h$

Dimostrazione: Si prova per induzione su h . Se $h=0$, l'albero consiste di un solo nodo, che è anche una foglia.

Supponiamo che il teorema sia vero per alberi 2-3 di altezza h .

Consideriamo un albero 2-3 T di altezza $h+1$. Sia T' l'albero ottenuto eliminando da T l'ultimo livello. Per le foglie f' e i nodi n' di T' , il teorema è vero (ipotesi induttiva). Poiché ogni foglia di T ha almeno due e al più tre figli in T , $2 \times 2^h \leq f \leq 3 \times 3^h$. Poiché $n=n'+f$, allora si ha la tesi.

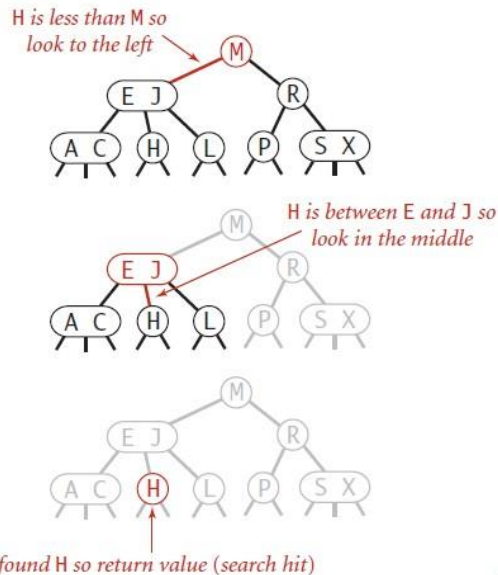
- Si deduce che $h=\Theta(\log n)$



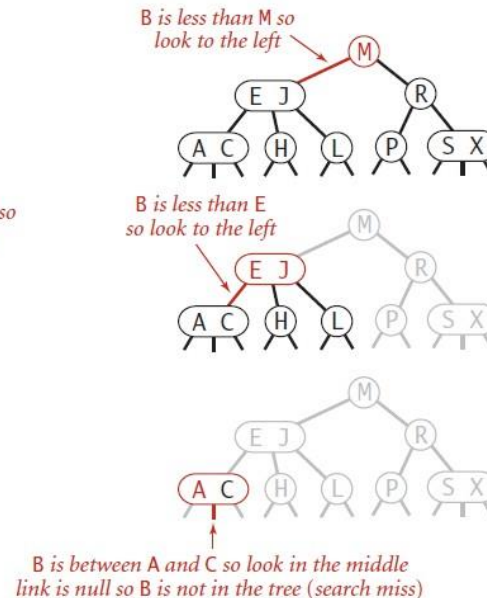
RICERCA IN UN ALBERO 2-3

- Usa una strategia simile a quella usata per i BST: si confronta la chiave da cercare con quelle contenute nel nodo. Se una è uguale si ha un successo, altrimenti si procede nel sottoalbero corrispondente. Se si trova un link nullo, si ha un insuccesso

successful search for H



unsuccessful search for B

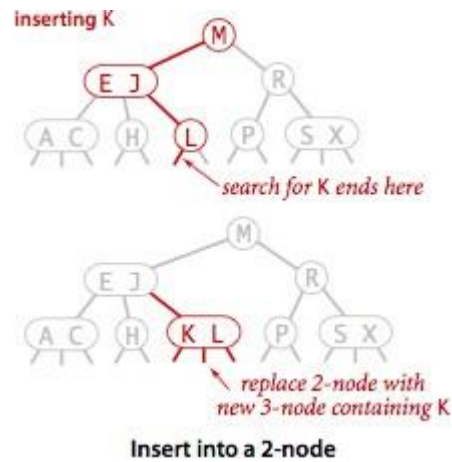


Search hit (left) and search miss (right) in a 2-3 tree



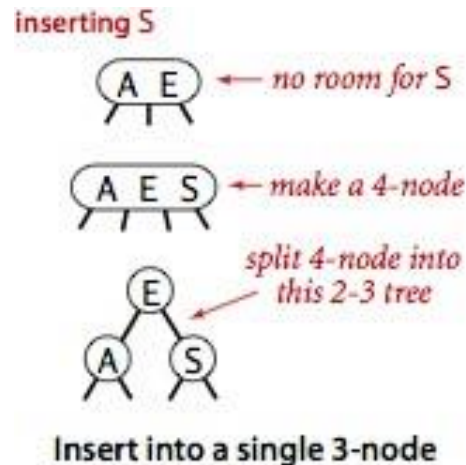
INSERIMENTO IN UN 2-NODO

- Come per i BST, si ricerca la posizione dove effettuare l'inserimento. Se la ricerca termina in un 2-nodo basta sostituire il 2-nodo con un 3-nodo che contiene anche la nuova chiave.



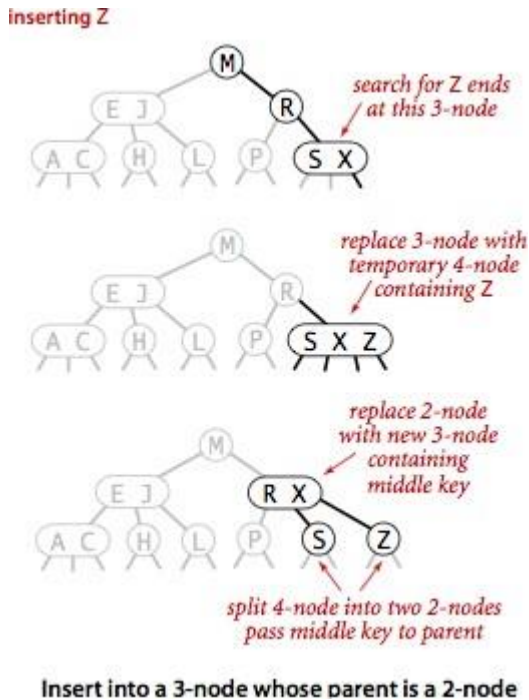
INSERIMENTO IN UN 3-NODO

- Inserimento in un singolo 3-nodo: Si pone temporaneamente la chiave nel nodo (diventa un 4-nodo). Si converte facilmente in un albero 2-3.



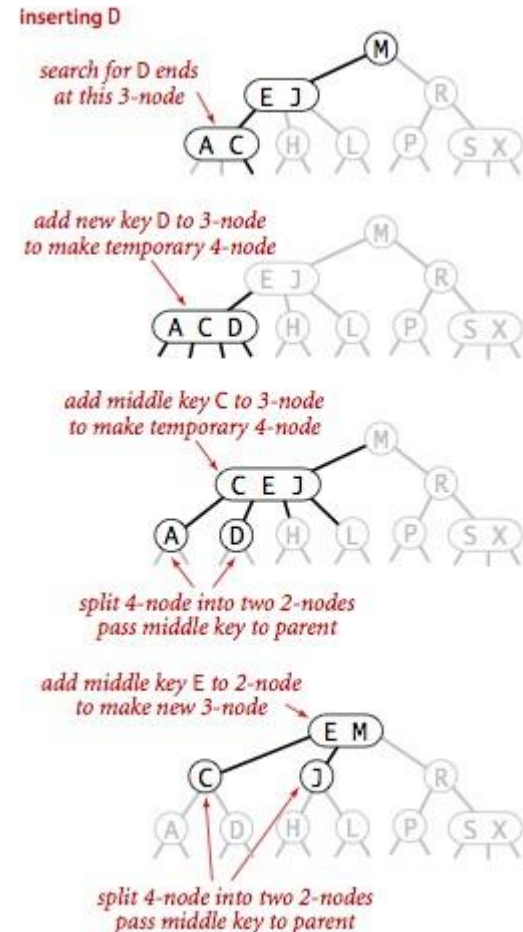
INSERIMENTO IN UN 3-NODO

- Inserimento in un 3-nodo il cui padre è un 2-nodo: Si pone temporaneamente la chiave nel nodo (diventa un 4-nodo). Si splitta il 4-nodo in modo tale che la chiave di mezzo si sposta sul padre



INSERIMENTO IN UN 3-NODO

- Inserimento in un 3-nodo il cui padre è un 3-nodo: Si pone temporaneamente la chiave nel nodo (diventa un 4-nodo). Si sposta la chiave di mezzo verso il padre (e così via, fino a che non si raggiunge un 2-nodo oppure la radice)

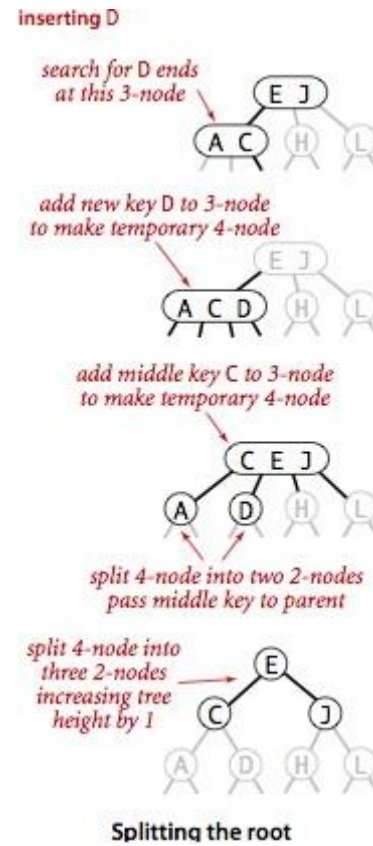


Insert into a 3-node whose parent is a 3-node



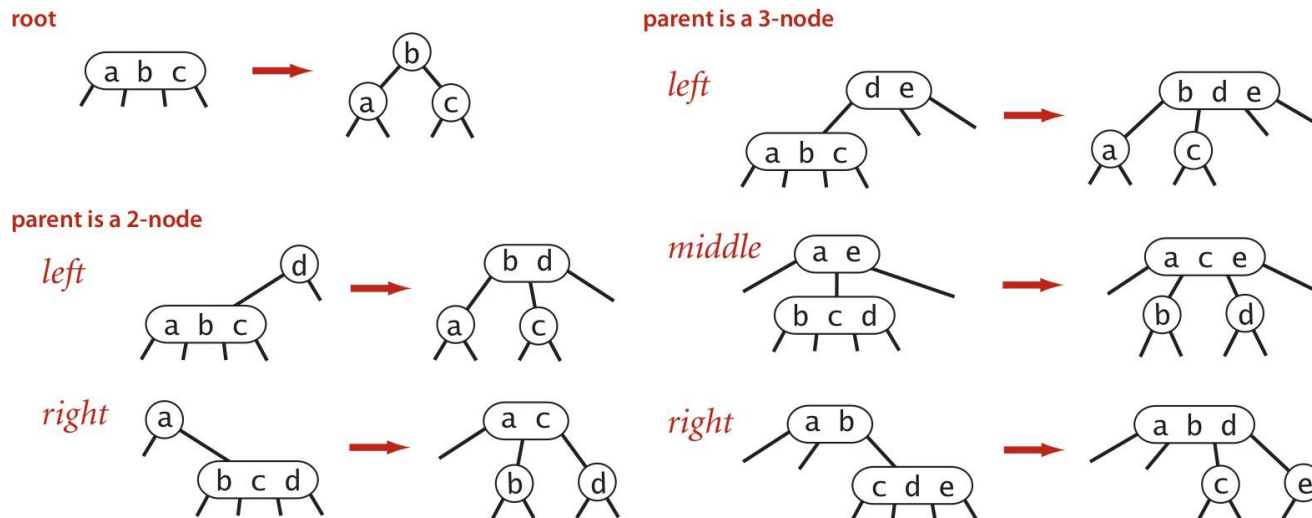
INSERIMENTO IN UN 3-NODO

- Split della radice: Si crea un 4-nodo provvisorio



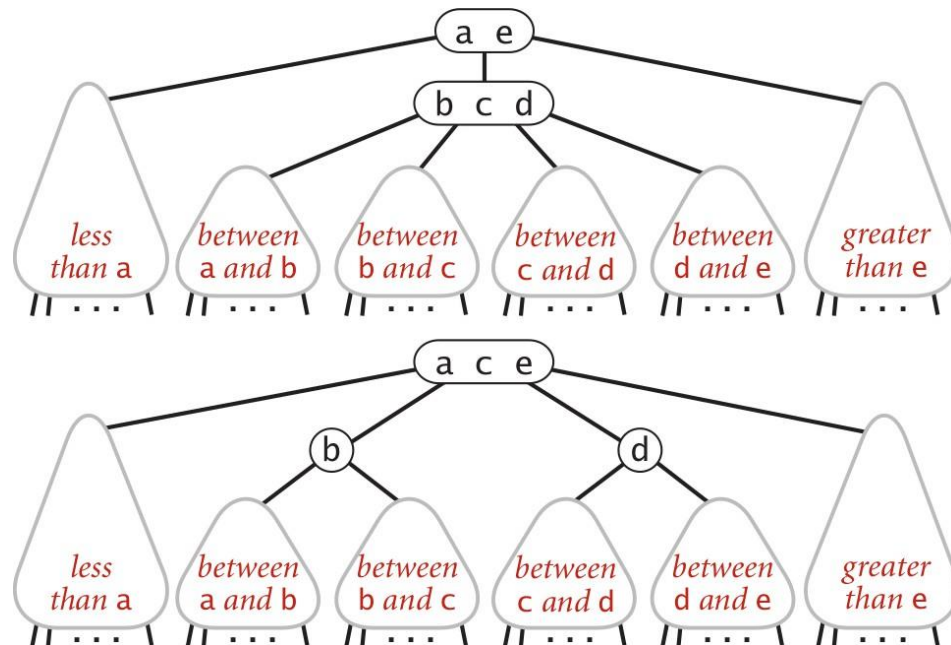
PROPRIETÀ DEI 2-3 ST

- Tutte le trasformazioni sono locali: nessuna altra parte dell'albero deve essere esaminata;
- Lo split di un 4-nodo comporta un numero limitato di trasformazioni. Il numero di cambiamenti di link è limitato da una costante piccola.



PROPRIETÀ DEI 2-3 ST

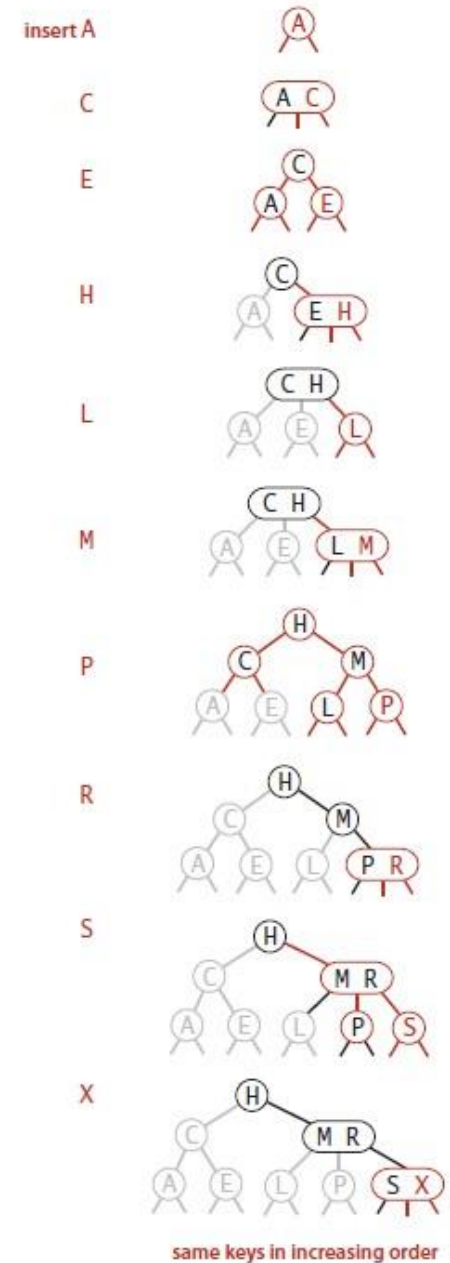
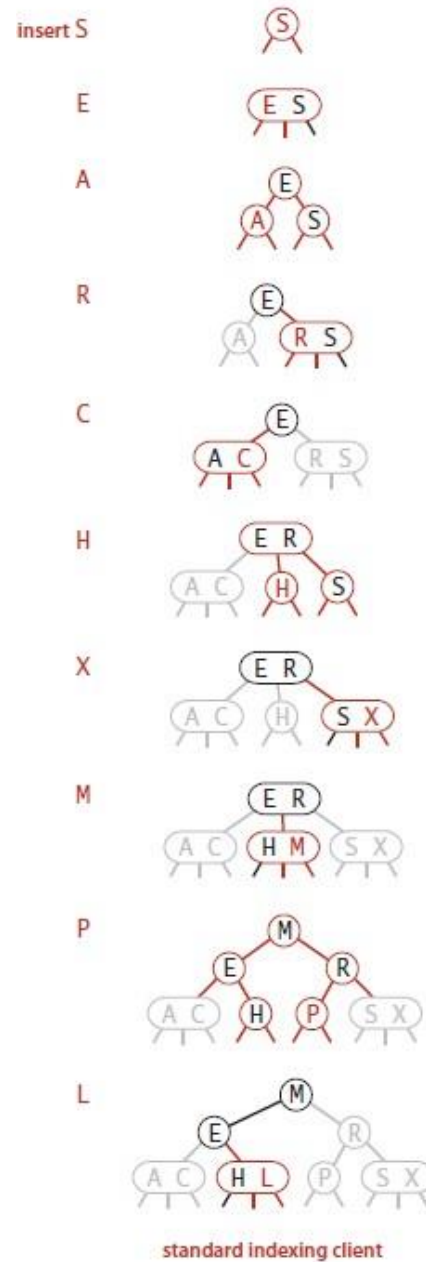
- Proprietà globali:
 - Invariante: l'albero è ordinato e perfettamente bilanciato;
 - Solo quando il 4-nodo alla radice viene diviso in tre 2-nodi si ha l'incremento di 1 per la lunghezza di tutti i cammini dalla radice ai link nulli.



ESEMPI

- Sequenza 1: S E A R C H X M P L
- Sequenza 2: A C E H L M P R S X

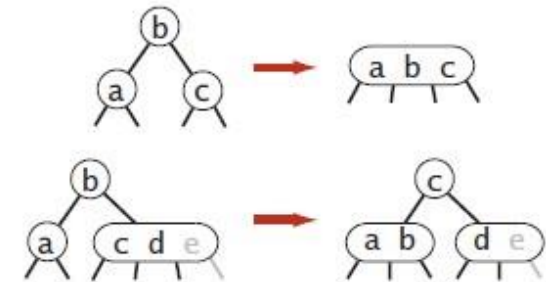
(confrontare con inserimento su BST)



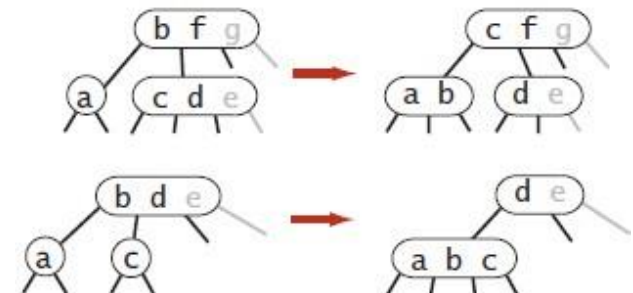
CANCELLARE IL MINIMO

- Cancellare una chiave da un 3-nodo che è una foglia è semplice. Non lo è se si tratta di un 2-nodo.
- Facciamo in modo di finire non in un 2 nodo da cancellare, applicando opportune trasformazioni.
 - Come si procede? Partendo dalla radice, si verifica se i figli sono entrambi 2-nodi. In tal caso si crea un 4-nodo temporaneo. Se il figlio di sinistra è un 2-nodo e il fratello no, si trasferisce un elemento dal fratello
 - Analogamente procedendo con il figlio di sinistra
 - In questo modo arriviamo ad una foglia che non è un 2-nodo.
 - Risalendo sistemiamo eventuali 4-nodi temporanei rimasti.

at the root



on the way down



at the bottom



CANCELLAZIONE DI UN NODO

- Lo stesso tipo di trasformazioni appena viste per la cancellazione del minimo si possono considerare quando si cerca un elemento da cancellare nell'albero 2-3 per assicurarsi che il nodo corrente non sia un 2-nodo.
- Se il nodo cercato è una foglia, si elimina. Se non è una foglia, si effettua come per i BST, una sostituzione con il suo successore (il minimo del sottoalbero di destra) applicando la cancellazione del minimo nel sottoalbero di destra.

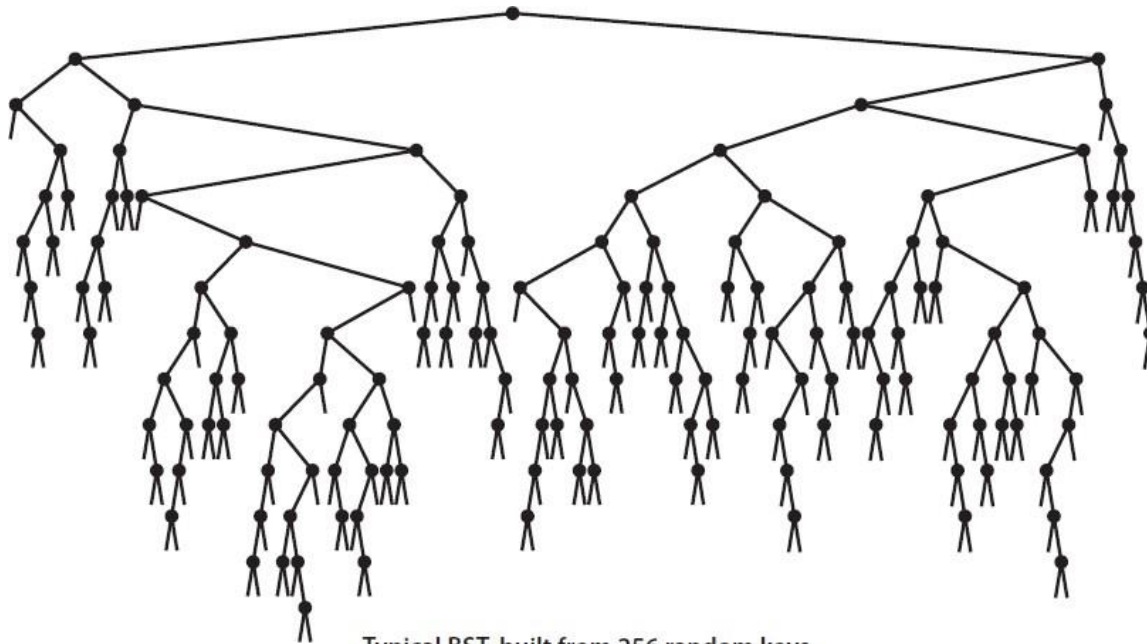


2-3 ST: ANALISI DELLE OPERAZIONI

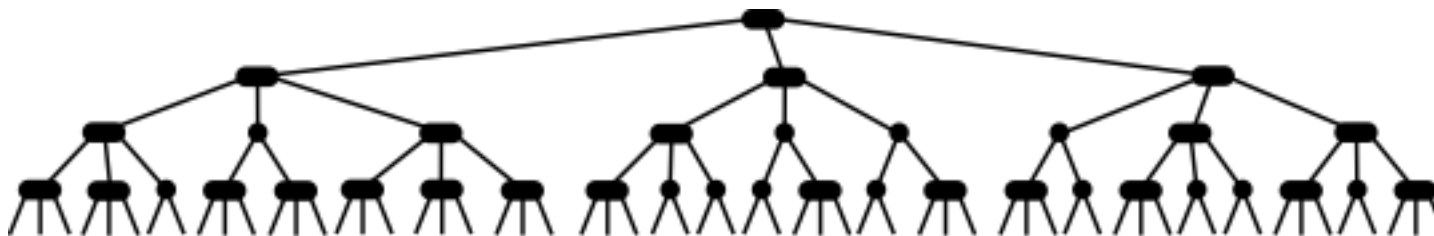
- Ogni operazione ad un nodo richiede tempo costante;
- Sia la ricerca sia l'inserimento esaminano nodi su un solo percorso tra radice e link nulli.
- Tutte le operazioni nel caso peggiore hanno un costo logaritmico rispetto al numero dei nodi.
- A differenza degli alberi AVL che potevano richiedere molte operazioni di rotazioni dopo un'operazione di rimozione, gli alberi 2-3 possono richiedere molte operazioni di split o fusione dopo un inserimento o rimozione



BST VS 2-3 ST (CON CHIAVI RANDOM)



Typical BST, built from 256 random keys



Typical 2-3 tree built from random keys



2-3 ST: IMPLEMENTAZIONE

- E' complicata:
 - mantenere diversi tipi di nodi;
 - comparazioni multiple per scendere nell'albero;
 - propagazione degli split verso l'alto;
 - molti casi diversi di splitting.

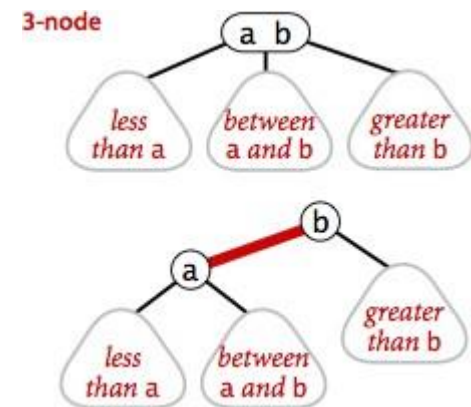




ALBERI ROSSO-NERI

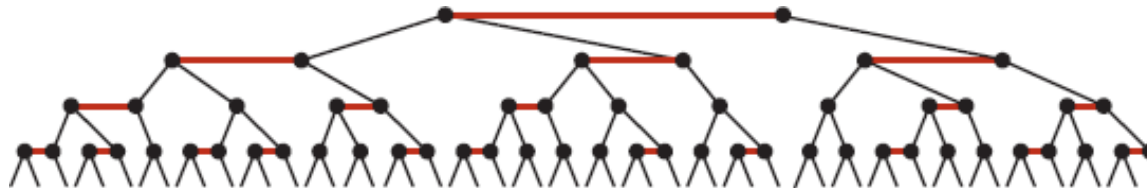
RED-BLACK BST

- Sono stati introdotti nel 1972 da Bayer. Guibas and Sedgwick introdussero la colorazione rosso-nero nel 1978. Una variante, gli **alberi rossi-a-sinistra-neri** è stata introdotta nel 2007 da Sedgwick.
- Rappresentano una particolare codifica dei 2-3 ST. Esistono altre varianti che codificano gli alberi (2,4)
- Ovvero, codifica degli alberi 2-3 a partire da BST, mediante aggiunta di informazione per codificare i 3-nodi, i quali sono rappresentati come coppie di 2-nodi, ciascuna coppia tenuta insieme da un arco rosso.
 - I nodi rossi sono orientati a sinistra;
 - Nessun nodo ha due archi rossi ad esso connessi;
 - L'albero ha bilanciamento nero perfetto: ogni cammino dalla radice al link nullo ha lo stesso numero di link neri



R-B BST E 2-3 ST

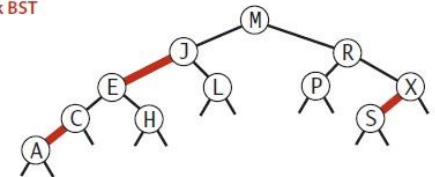
- C'è una corrispondenza 1-1 tra gli alberi rosso-neri definiti in questo modo e gli alberi 2-3



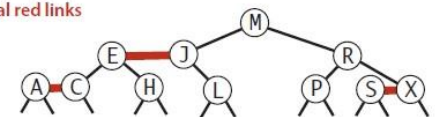
A red-black tree with horizontal red links is a 2-3 tree

- ricerca efficiente (la stessa di BST);
- inserimento con bilanciamento (lo stesso dei 2-3ST).

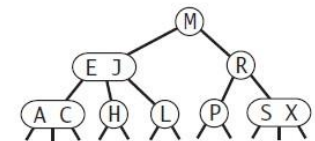
red-black BST



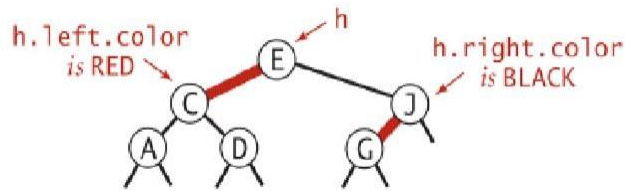
horizontal red links



2-3 tree



COME SI RAPPRESENTANO I NODI



Ad ogni nodo è associato il colore del link dal padre.

Per convenzione i link nulli sono BLACK

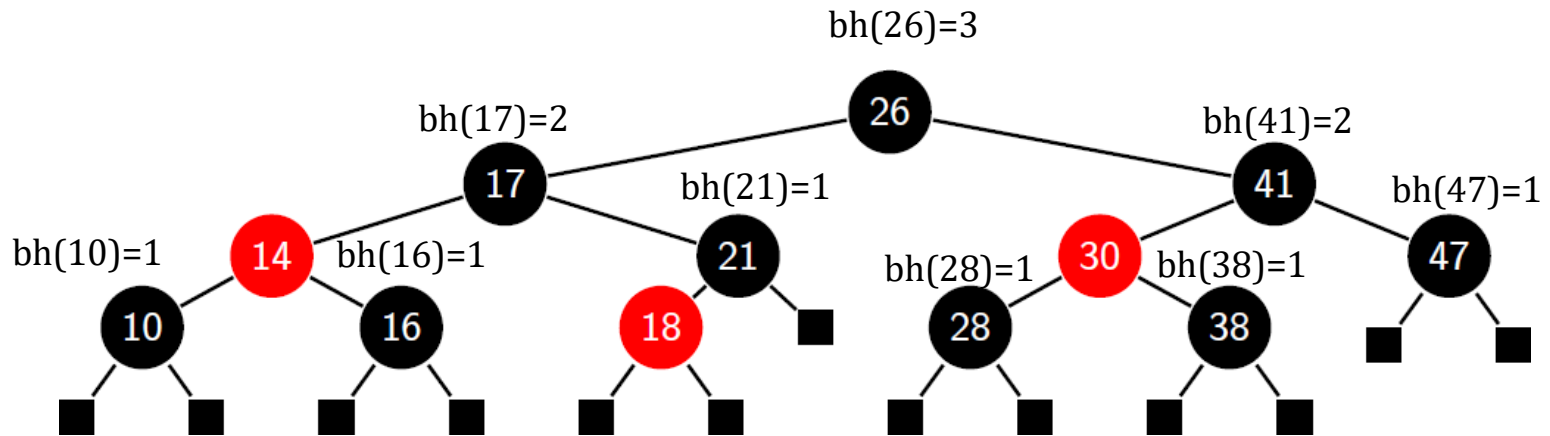
Con tale colorazione (**nella definizione classica più generale**):

- Ogni nodo ha colore rosso o nero
- La radice è nera
- Ogni foglia è nera, non ha contenuto informativo e contiene un link nullo
- Se un nodo è rosso, entrambi i suoi figli sono neri (ovvero ciascun nodo rosso deve avere padre nero)
- Ogni cammino da un nodo ad una foglia contiene lo stesso numero di nodi neri.
- **Nella variante rosso a sinistra-nero, un nodo nero non può avere due figli rossi.**



ESEMPI

📌🎯 Indichiamo con $bh(v)$ il numero di nodi neri in un qualsiasi cammino da v a una foglia (escluso v)

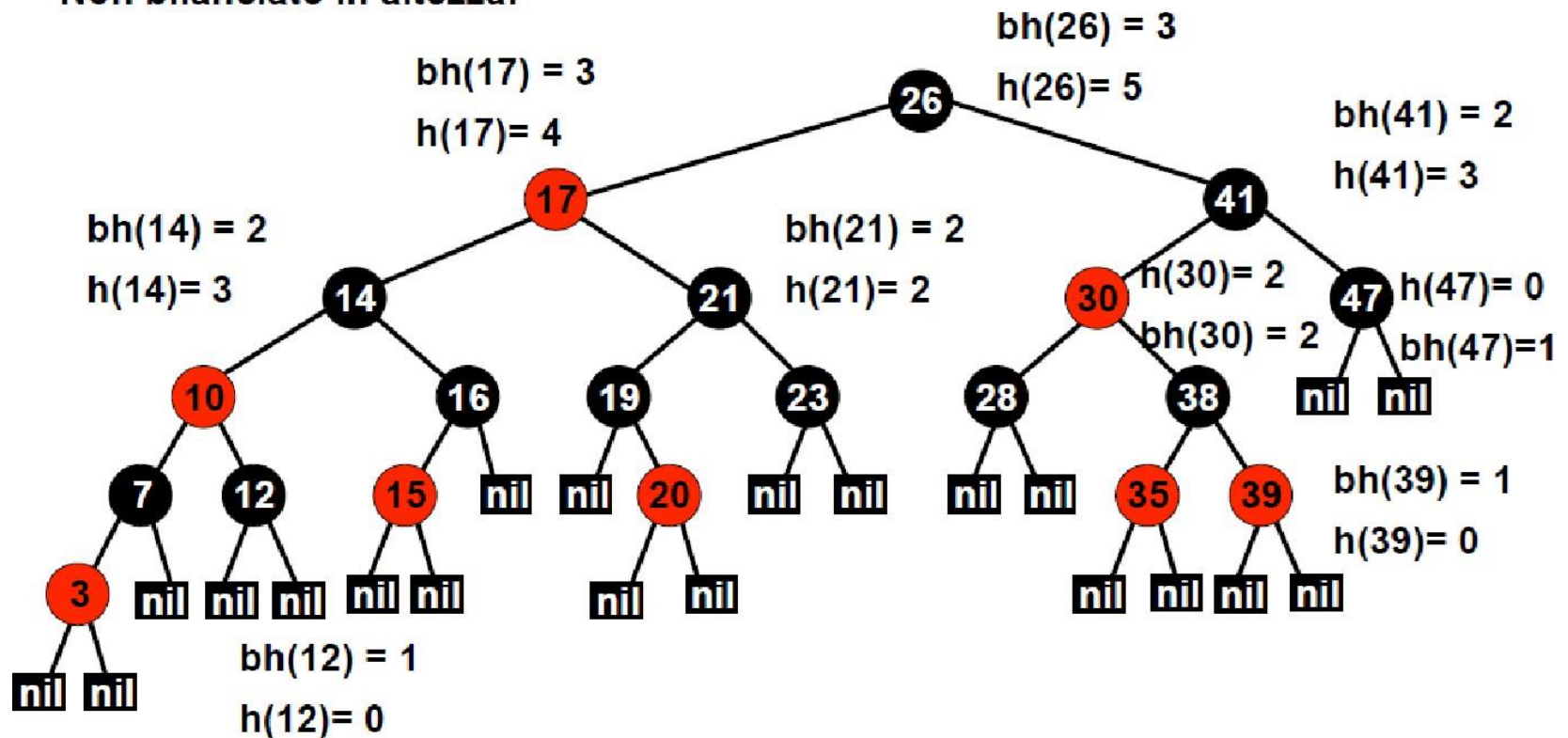


📌🎯 Nota che nella variante classica, un nodo nero può avere due figli rossi, o anche un figlio rosso a destra. Ciò codifica gli alberi (2,4)



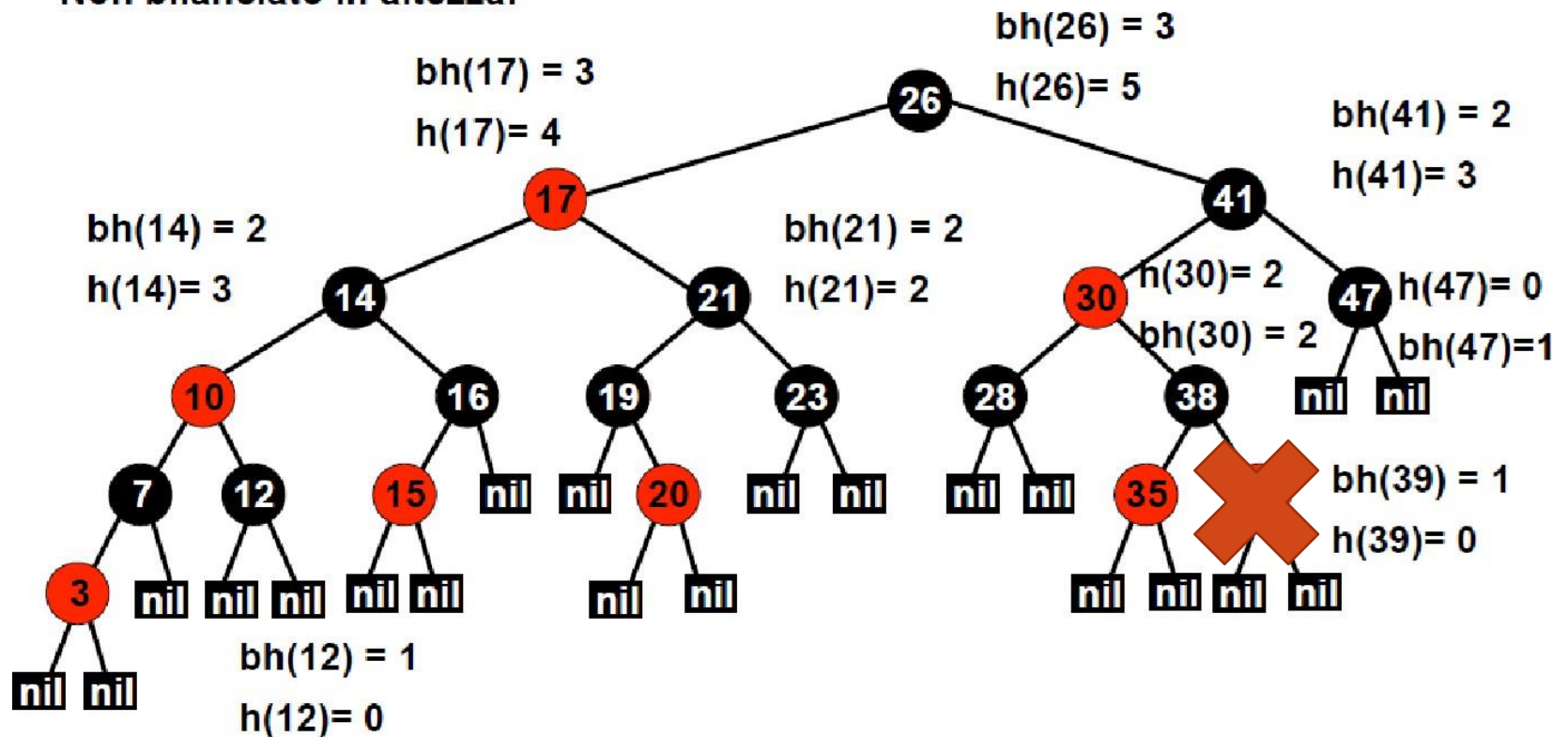
ESEMPIO DI ALBERO ROSSO-NERO

Non bilanciato in altezza!

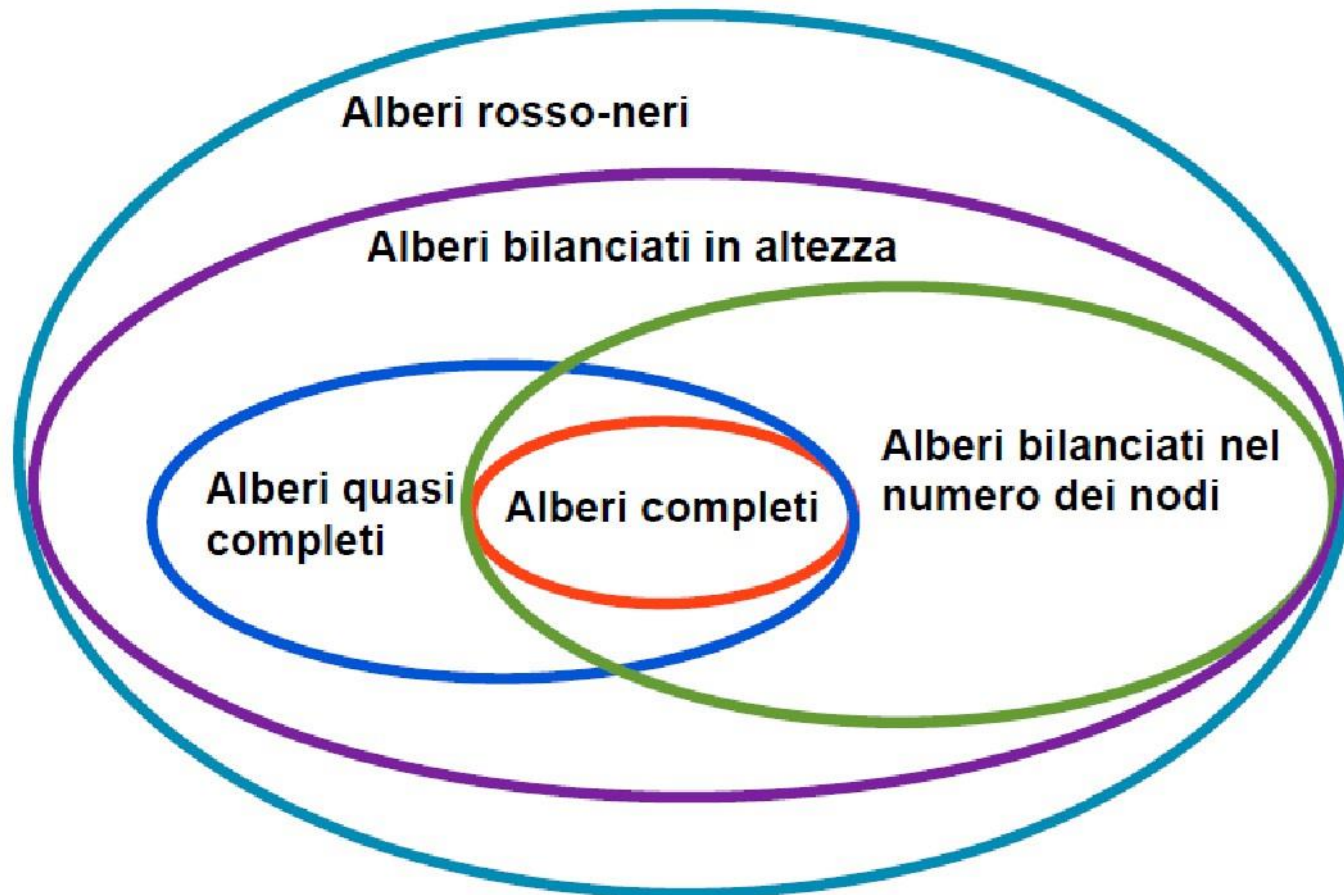


ESEMPIO DI ALBERO ROSSO-NERO

Non bilanciato in altezza!



RELAZIONE TRA CLASSI DI ALBERI BILANCIATI



PROPRIETÀ DI UN ALBERO ROSSO-NERO

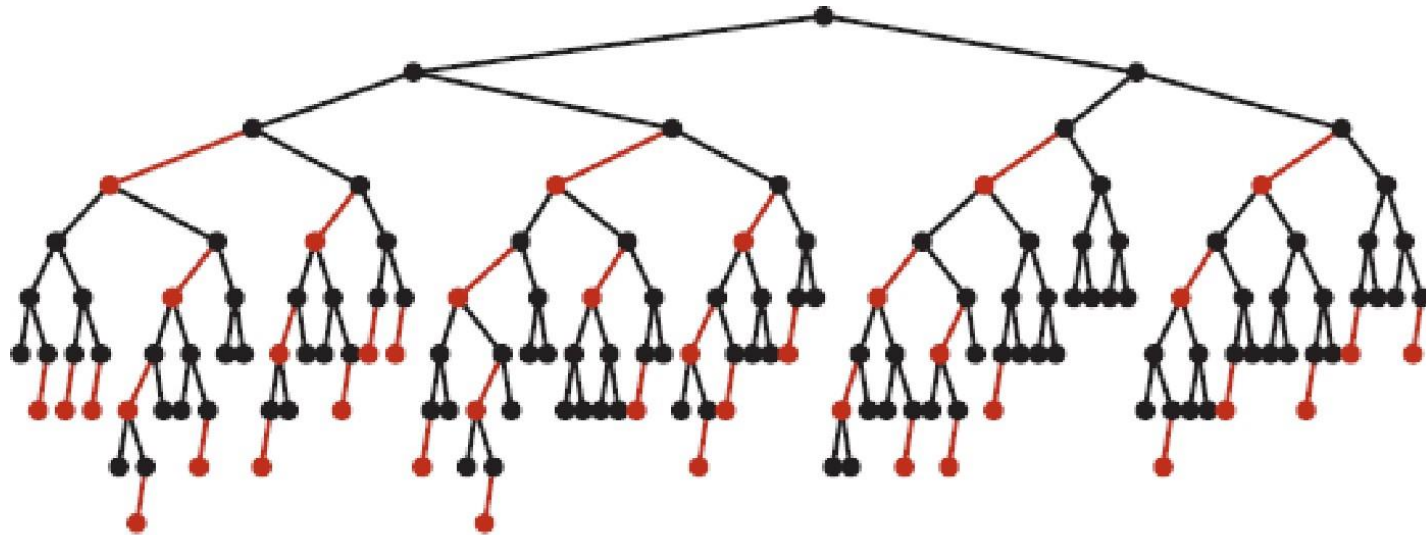
📌🎯 L'altezza di un albero di ricerca RB con N nodi è $\Theta(\log N)$

Idea della dimostrazione: Il caso peggiore è un albero 2-3 i cui nodi sono tutti 2-nodi tranne il percorso più a sinistra fatto di 3-nodi. Il percorso che è costituito dai link sinistri è due volte più lungo rispetto a quello che coinvolge i 2-nodi che è $\sim \log N$ (*lower bound*). Quindi è possibile creare alberi rosso neri con un cammino di lunghezza $2 \log N$ (*upper bound*).



R-B BST CON CHIAVI RANDOM

📌🎯 In un RB BST con N nodi, la distanza media di un nodo dalla radice è circa $1.00 \lg N$.



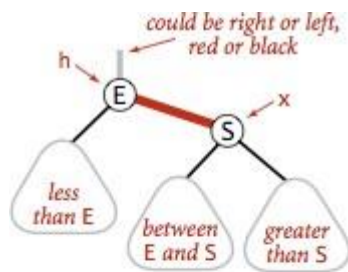
Typical red-black BST built from random keys (null links omitted)



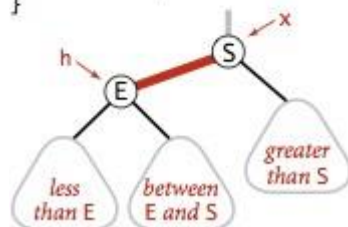
ROTAZIONI

(NELLA VARIANTE ROSSI A SINISTRA-NERI)

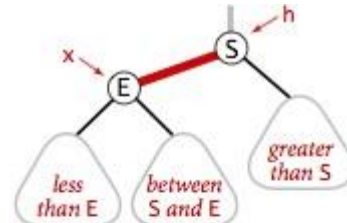
📌🕒 In via transitoria i link rossi possono tendere a destra



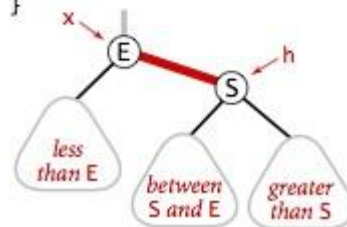
```
Node rotateLeft(Node h)
{
    Node x = h.right;
    h.right = x.left;
    x.left = h;
    x.color = h.color;
    h.color = RED;
    x.N = h.N;
    h.N = 1 + size(h.left)
        + size(h.right);
    return x;
}
```



Left rotate (right link of h)



```
Node rotateRight(Node h)
{
    Node x = h.left;
    h.left = x.right;
    x.right = h;
    x.color = h.color;
    h.color = RED;
    x.N = h.N;
    h.N = 1 + size(h.left)
        + size(h.right);
    return x;
}
```



Right rotate (left link of h)

rotateLeft() e
rotateRight() mantengono le
invarianti:

Ordine simmetrico;
Perfetto bilanciamento nero.



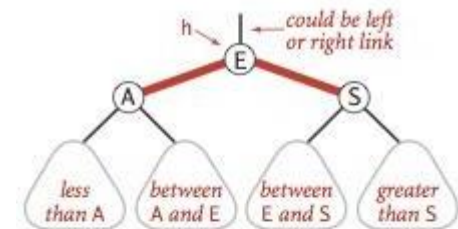
COMMUTAZIONE DEI COLORI (COLOR FLIP)

🚩🎯 In via transitoria, due archi rossi possono insistere sullo stesso nodo. Per dividere un 4-nodo può essere necessario ricolorarlo.

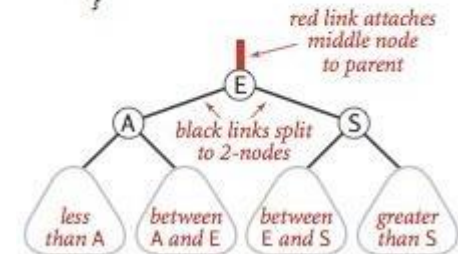
🚩🎯 mantiene le invarianti:

🚩🎯 Ordine simmetrico;

🚩🎯 Perfetto bilanciamento nero.



```
void flipColors(Node h)
{
    h.color = RED;
    h.left.color = BLACK;
    h.right.color = BLACK;
}
```



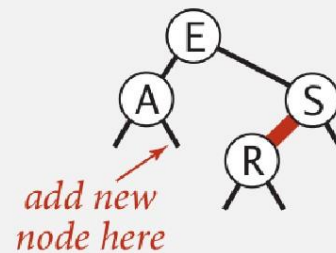
Flipping colors to split a 4-node



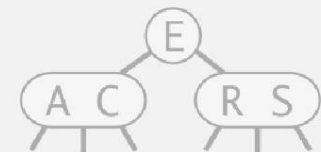
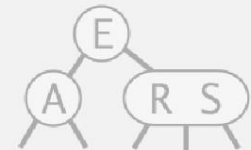
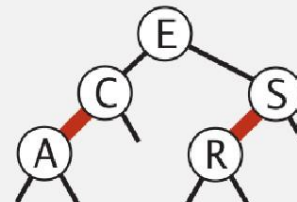
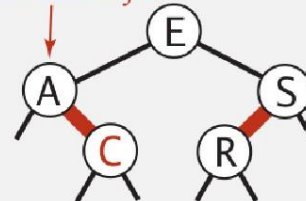
INSERIMENTO: STRATEGIA

- 📌🎯 Mantenere la corrispondenza con gli alberi 2-3

insert C

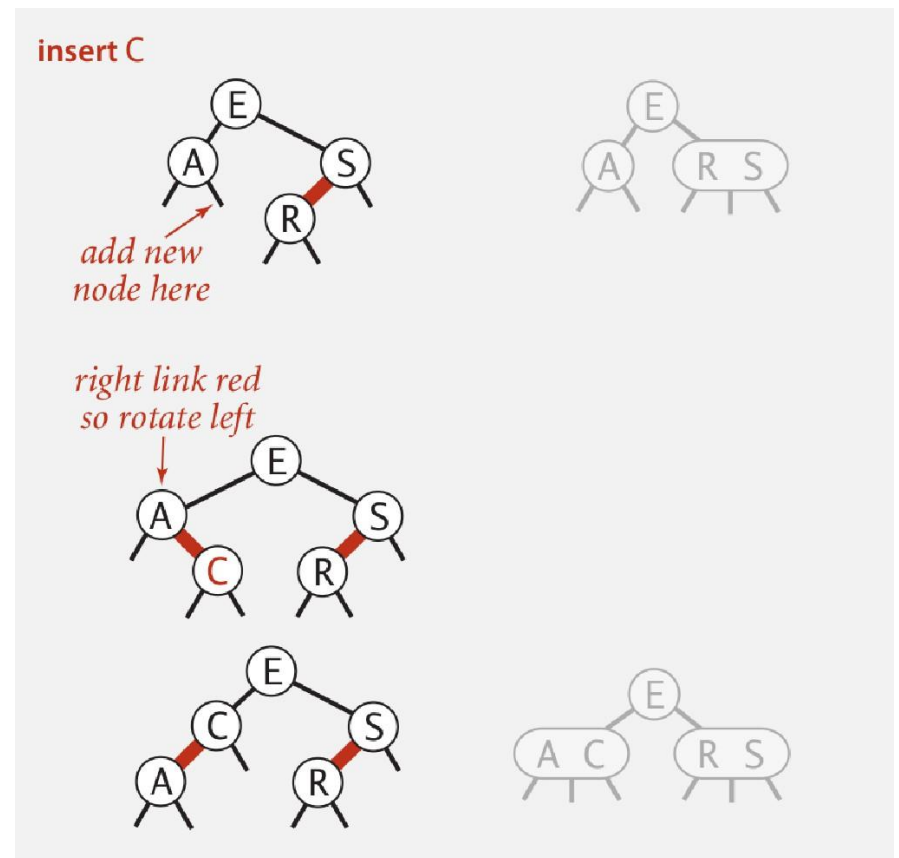


right link red
so rotate left



INSERIMENTO IN 2-NODO

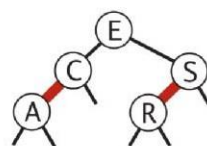
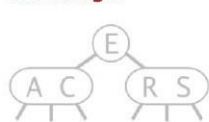
- ❗🎯 Fai inserimento BST standard.
- ❗🎯 Colora di rosso il nuovo link;
- ❗🎯 se il nuovo link è destro, fai una rotazione a sinistra.



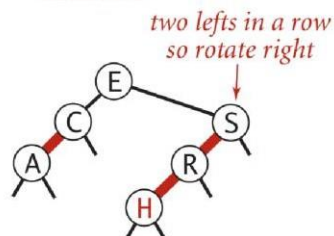
INSERIMENTO IN 3-NODO

- ❗🎯 Fai inserimento BST standard. Colora di rosso il nuovo link;
- ❗🎯 se necessario, ruota per bilanciare il 4-nodo;
- ❗🎯 commuta i colori per passare il rosso verso l'alto;
- ❗🎯 se necessario, ruota per far tendere il rosso a sinistra.

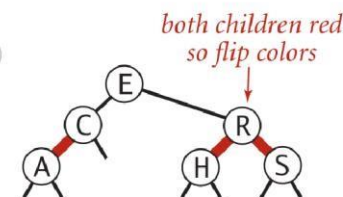
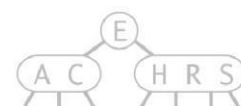
inserting H



add new node here

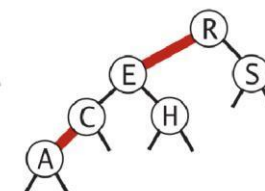
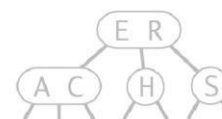
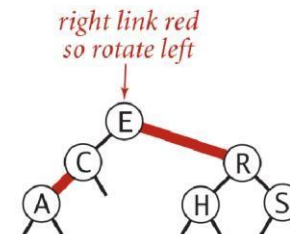


two lefts in a row
so rotate right

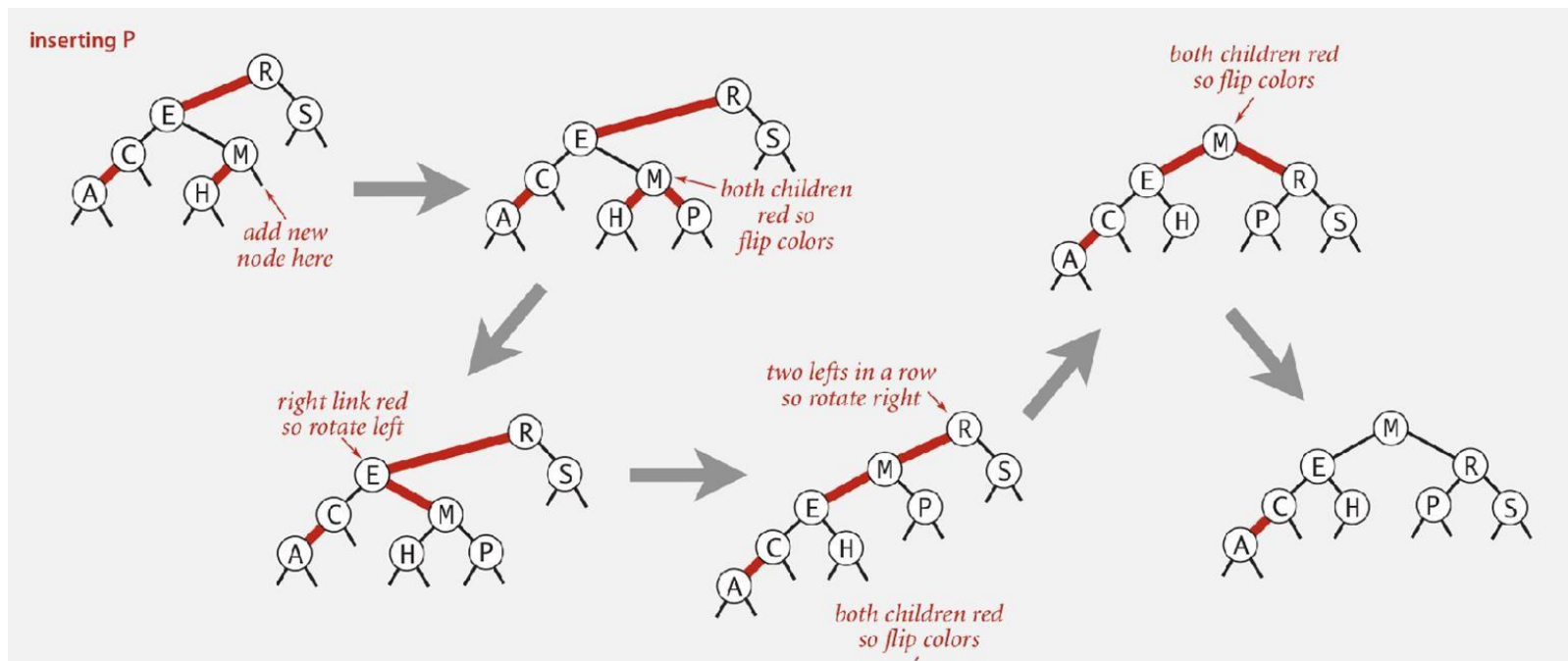


both children red
so flip colors

right link red
so rotate left



PROPAGAZIONE DEGLI ARCHI ROSSI VERSO L'ALTO

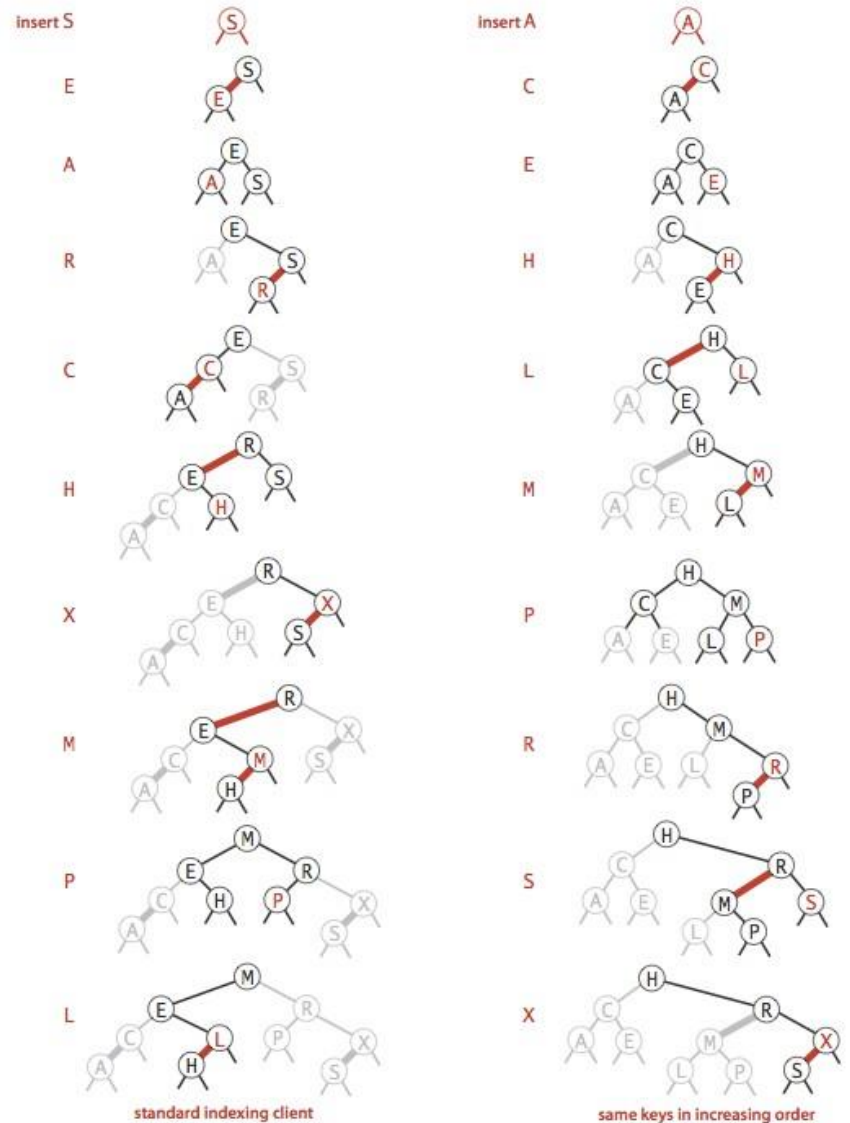


ESEMPI

🚩🎯 Sequenza 1: S E A R C H X M P L

🚩🎯 Sequenza 2: A C E H L M P R S X

(confrontare con inserimento su BST)



Red-black BST construction traces



OPERAZIONI A CONFRONTO

algoritmo (struttura dati)	caso peggiore (dopo N inserzioni)		caso medio (dopo N inserzioni casuali)		operazioni ordinate efficienti?
	search	insert	search hit	insert	
ricerca se- quenziale (lista concatenata non ordinata)	N	N	$N/2$	N	no
ricerca binaria (array ordinato)	$\log N$	$2N$	$\log N$	N	si
albero binario di ricerca (BST)	N	N	$1.39 \log N$	$1.39 \log N$	si
albero 2-3 (RBBST)	$2 \log N$	$2 \log N$	$1.00 \log N$	$1.00 \log N$	si



RELAZIONE TRA ALBERI ROSSO-NERI E AVL

- ❏ © Gli alberi rosso-neri sono i migliori quando le sequenze di elementi inseriti sono costruite a caso (la regola di bilanciamento per gli alberi rosso-neri è più permissiva di quella degli AVL quindi si fanno meno operazioni di ribilanciamento) con occasionali inserimenti di sequenze di elementi ordinati, se invece gli inserimenti di sequenze di elementi ordinati sono prevalenti allora sono gli AVL a comportarsi meglio.
- ❏ © Se le sequenze di elementi inseriti sono costruite a caso a comportarsi meglio sono i semplici BST, ma se i dati sono inseriti in ordine allora i BST degenerano in liste.



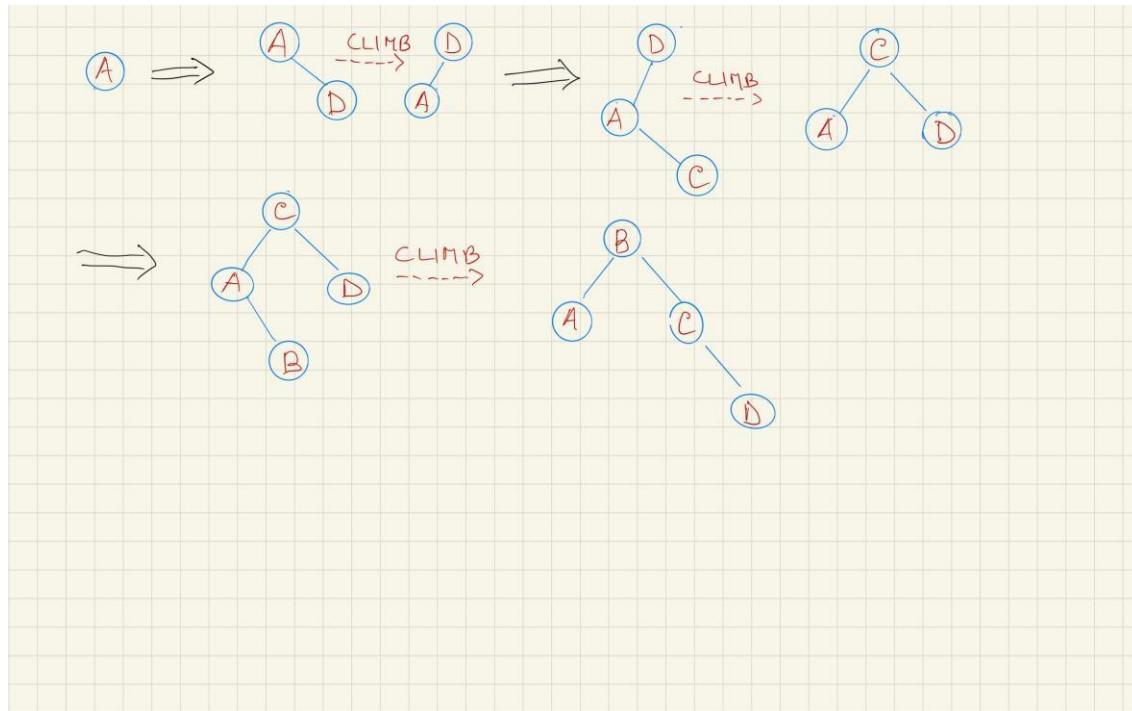
ESERCIZIO

- Si definisca una classe che implementi un particolare albero binario di ricerca (che contiene caratteri come contenuti informativi), chiamato Fog Search Tree (FST) di ricerca in cui ogni volta che una chiave K viene inserita o cercata nell'albero, essa viene spostata nella radice (mantenendo la proprietà di essere un BST), attraverso una successione di operazioni di risalita chiamate CLIMB.
 - Definire in modo opportuno ed efficiente il/i metodo/i necessari a realizzare le CLIMB operations, sia quando un elemento viene cercato nell'albero sia quando viene inserito, che quando viene cancellato. In particolare,
 - se l'elemento K viene cercato nel FST e viene trovato in un dato nodo N su questo viene applicata la CLIMB operation. Se l'elemento K non è presente, l'operazione viene applicata sulla foglia dove termina la ricerca
 - se si inserisce l'elemento K, su tale elemento si applica l'operazione di CLIMB
 - se si rimuove un nodo, l'operazione di CLIMB viene applicata sul nodo genitore del nodo rimosso
 - Quali sono le complessità di tempo di esecuzione delle operazioni di dizionario nel caso peggiore?



POSSIBILE ESEMPIO DI FST CON CLIMB OPERATIONS

- Inserimento degli elementi A, D, C, B



ESERCIZIO

Eseguire le seguenti operazioni su un albero inizialmente vuoto e testare il numero delle operazioni CLIMB e il costo totale delle operazioni dopo ciascun set

- Insert 0, 2, 4, 6, 8, 10, 12, 14, 16, 18, in quest'ordine.
- Search 1, 3, 5, 7, 9, 11, 13, 15, 17, 19, in quest'ordine.
- Delete 0, 2, 4, 6, 8, 10, 12, 14, 16, 18, in quest'ordine.

